



S.G. Ganesh

Misconceptions About Java

I find that many Java programmers, both new and experienced, have misconceptions about the language. We'll examine some of these in this column.

Can you tell which of these statements is true about Java?

- 1) Resource leaks cannot occur in Java; unlike C/C++, the garbage collector (GC) will take care of collecting all resources, including memory.
- 2) Java programs are fully portable; unlike C/C++, a Java program will run exactly the same way, no matter which platform it is run on.
- 3) Thread programming is easy in Java; unlike C/C++, Java has multi-threading support in the language (in addition to library support), so it's easy to write correct concurrent code.
- 4) Java has a better type system than languages like C/C++; the kind of type-related mistakes you make in C/C++ just cannot happen in Java—and you don't have to know machine-level details when programming in Java.
- 5) Java is easy to master; it just takes a couple of weeks since it does not have complexities like C/C++.

The answer is simple: **none of these statements are true!** I find that Java is often compared with C/C++, so I've deliberately selected these statements to dispel some myths about Java. Let's discuss each of these statements in detail.

- 1) Yes, memory leaks in the conventional C/C++ sense don't occur in Java, but 'logical memory leaks' can happen. For example, when a field retains a reference to an object even after its use is over, the GC cannot collect that object, since it is 'live' from the GC's perspective. Sloppily written Java applications often crash with *OutOfMemoryError* because of such leaks. Also, the GC only takes care of memory; it is the programmer's responsibility to release all other resources (file handles, database connections, etc). Trying to release such non-memory resources in *finalize* methods (with no assurance about if and when they'll run) is the cause of another set of headaches in Java programs.
- 2) Yes, Java is certainly more portable than C/C++ programs. However, to say that Java programs are fully portable is far from the truth: Java makes it easy to write portable programs, but Java programs are not automatically portable. Even simple differences like how we specify file paths (the path separator is "/" on Linux and "\" on Windows) can make a Java program non-portable. Subtle

issues like multi-threading issues can make a real-world Java application very difficult to port.

- 3) Yes, Java has threading support at the language level, like the *synchronized* keyword. It also has support in the library—for example, the *java.lang.Thread* class. But anyone who has written non-trivial multi-threaded code in real-world Java applications will appreciate the fact that it is extremely difficult to write 'correct' multi-threaded programs. Concurrent programming is inherently difficult; what's fair to say is that Java has somewhat better support than in C/C++! Moreover, multi-threading support differs from platform to platform, and subtle differences in JVM implementations make porting multi-threaded Java programs very difficult.
- 4) Try answering this simple Java question: what is the value of *a* after the last line of this code snippet?

```
int x= 255;
byte y = (byte)x;
char z = (char)y;
int a = (int)z;
```

The correct answer is that the value of *a* is 65535! When you ask expert Java programmers why this happens, their explanation will cover sign-extension, 2's complement representation, subtle rules about widening conversions from Java language specifications, etc. On the surface, it would appear that Java has more type safety than C/C++ (for example, Java disallows implicit conversion from the *int* type to *boolean*); but underneath, it is similar to C/C++ with its complicated rules.

- 5) Just try answering questions in the excellent book 'Java puzzlers: Traps, pitfalls, and corner cases' by Joshua Bloch and Neal Gafter. It takes years to master a language, and Java is no exception. As they say, 'Nothing good ever comes easy!' 

By: S.G. Ganesh

The author works in Siemens (Corporate Technology), Bengaluru. His latest book is the best-seller *Cracking the C, C++ and Java Interview*, published by Tata McGraw-Hill. You can reach him at sgganesh@gmail.com.